

Course: CSE302 Computer Organization and Architecture

Group Project: Design a five-cycle non-pipelined RISC processor using Verilog HDL

- Your task is to design a five-cycle non-pipelined RISC processor using Verilog HDL. You will use the Logisim based processor design provided in the class as the reference, and as a means for understanding the design, doing comparison of results (functional-verification/validation) during testing.
- Note, in the Verilog HDL design of the processors, all FFs (FF based modules or clk based memory) will have Reset
- Precisely follow the names of the modules and the ports given here.

1. Create processor.v file with the following module

Module Name	Inputs	Outputs	Comments
RISCprocessor	clk Reset [7:0] InpExtWorld1 [7:0] InpExtWorld2 [7:0] InpExtWorld3 [7:0] InpExtWorld4	[7:0] OutExtWorld1 [7:0] OutExtWorld2 [7:0] OutExtWorld3 [7:0] OutExtWorld4	(Pure Structural coding) There will be several modules instantiated within this module; Some importance instances are mentioned below: RegisterFile InstMEM SRAM Stack ALU TimingGen ProgCounter INport OUTport ControlLogic & some MUX32to1 8bit

2. Create subcomponents.v file with the following modules [Any smaller modules not mention in the table below, will have to be created by you – that is implicit]

Module Name	Inputs	Outputs	Comments/Guidance
eightbitRegwithLoad	clk Reset load [7:0] Datain	[7:0] Dataout	Code already available with you in Lab5 Remember that, this module can also be converted to (or used as) an 8-bit FF by simply passing 1'b1 to load when instantiating it in another module
RegisterFile	clk Reset RegFileRead RegFileWrite [7:0] Datain [3:0] Source1 [3:0] Source2 [3:0] Destin	[7:0] Dataout1 [7:0] Dataout2	Structural coding preferred, with some important instances: 16 instances of eightbitRegwithLoad 1 instance of Decoder4to16_withE 2 instances of MUX16to1_8bit 2 instances of eightbitRegwithLoad
InstMEM	clk Reset [7:0] Address InstRead	[24:0] Dataout [4:0] Opcode [3:0] Destin [3:0] Source1	(Behavioral coding) The memory variable would be declared as:

		[3:0] Source2 [7:0] Imm	reg [24:0] instmemory [0:255] Use initial block and \$readmemb to load your assembly language program (instructions in binary format)
SRAM	clk Reset [7:0] Address SRAMRead SRAMWrite [7:0] Datain	[7:0] Dataout	(Behavioral coding) The memory variable would be declared as: reg [24:0] datamem [0:255]
Stack	clk Reset [7:0] Address StackRead StackWrite [7:0] Datain	[7:0] Dataout	Partial structural coding, with the following important instances: 1 instance of SRAM 2 instances of MUX4to1_8bit And some more instances of relevant modules to create the stack pointer/counter logic
ALU	clk Reset Imm7 [7:0] Operand1 [7:0] Operand2 Opcode ALUSave ZflagSave CflagSave	Zflag Cflag [7:0] ALUout	Any style
TimingGen	clk Reset	T0 T1 T2 T3 T4	Any style
ProgCounter	clk Reset PCenable PCupdate [7:0] CAddress	[7:0] PC [7:0] PC_D2	Any style
INport	clk Reset INportRead [7:0] InpExtWorld1 [7:0] InpExtWorld2 [7:0] InpExtWorld3 [7:0] InpExtWorld4 [7:0] Address	[7:0] Dataout	Any style
OUTport	clk Reset [7:0] Address [7:0] Datain OUTportWrite	[7:0] OutExtWorld1 [7:0] OutExtWorld2 [7:0] OutExtWorld3 [7:0] OutExtWorld4	Any style
ControlLogic	clk Reset T1	PCupdate SRAMRead SRAMWrite	Structural coding preferred, with some important instances:

	T2 T3 T4 Zflag Cflag [4:0] Opcode	StackRead StackWrite ALUSave ZflagSave CflagSave INportRead OUTportWrite RegFileRead RegFileWrite	Each control signal would be implemented using an instance of MUX32to1_1bit_withE
--	--	---	---

- Each group will prepare one powerpoint based presentation (worth 10 minutes), and present their work using the projector in Lab 205, as per the slots (to be announced) on the dates (already in the list of lectures/lab) in April 2025.
 - o Each person of the group will have to speak.
 - o It will be the responsibility of the group to check beforehand (1-2 days in advance) the appropriate connection (display settings and any adapters needed) of their laptop for the presentation.
- Each group will submit the following files in 1st week of April 2025:
 - o **subcomponents.v**
 - o **processor.v**
 - o The text file containing their assembly language code (in binary format)
 - (corresponding) hand-written assembly language code that was tested
 - o Powerpoint presentation

The end